# 软件需求分析与设计

## 第三章框架梳理

仅供学习参考

# 目 录


**一、 用例视图（Use Case View）- 1 -**

（一） 用例（Use Case）- 1 -

1. 定义及性质- 1 -

（二） 参与者（Actor）- 1 -

1. 定义及性质- 1 -

（三） 用例图（Use Case Diagram）- 1 -

1. 定义及性质- 1 -

（四） 用例文档化（Documenting use cases）- 1 -

1. 用例文档的结构- 1 -

**二、 活动视图（Activity View）- 2 -**

（一） 活动建模（Activity modeling）- 2 -

1. 活动模型的定义- 2 -

（二） 动作（Action）- 2 -

1. 动作的定义- 2 -

（三） 活动图（Activity diagram）- 2 -

1. 活动图的定义- 2 -

2. 活动图的表示- 2 -

**三、 结构视图（Structure View）- 3 -**

（一） 结构视图- 3 -

1. 定义及性质- 3 -

（二） 类（Class）- 3 -

1. 类的分类- 3 -

（三） 类图（Class diagram）- 3 -

1. 定义及性质- 3 -

2. 类之间的关系- 3 -

**四、 交互视图（Interaction View）- 4 -**

（一） 交互视图- 4 -

1. 交互视图的定义和性质- 4 -

（二） 顺序图（Sequence diagram）——对象交互过程的时间顺序- 4 -


仅供学习参考

1. 顺序图的组成 - 4 -
（三） 通信图（Communication diagram）——对象之间的协作关系 - 4 -
1. 通信图的性质 - 4 -
（四） 类方法 - 4 -
1. 消息与方法的关系 - 4 -
**五、 状态机视图（State machine view） - 5 -**
（一） 状态机模型 - 5 -
1. 定义和性质 - 5 -
（二） 状态机图 - 5 -
1. 定义和性质 - 5 -
2. 状态机图的组成元素 - 5 -
**六、 实现视图（Implementation View） - 6 -**
（一） 子系统和包 - 6 -
1. 子系统 - 6 -
2. 包 - 6 -
3. 区别与联系 - 6 -
（二） 构件和构件图 - 6 -
1. 构件（Component） - 6 -
2. 构件图 - 6 -
（三） 节点和部署图 - 6 -
1. 节点 - 6 -
2. 部署图 - 7 -

仅供学习参考

# 一、用例视图（Use Case View）

## （一）用例（Use Case）

1. 定义及性质

(1) 描述系统行为，所有用例都**直接或间接**的对某一个或几个参与者相关联

(2) 是参与者从**外部（outwardly）**可以看到的业务功能，完整的功能单元

(3) 用例定义主题所提供的行为，而**不需要**引用主题的内部结构

(4) 在很多情况下，一个功能性需求可以直接**映射（Map）**到一个用例

(5) 用例驱动整个软件开发的生命周期，是大多数开发活动的**焦点**和**参照**

(6) 在 UML 中使用**椭圆**  表示，名字放在椭圆的里面或下面

## （二）参与者（Actor）

1. 定义及性质

(1) 主题**外部的人**或事物针对用例所扮演的角色，即与用例进行**交互**的人或事物

① 参与者并不是某人或者某事的特定实例，而是描述**一类角色**（人、事物）

② 参与者通过诸如交换信号和数据的方式与主题**交流信息**，接收有用的结果

(2) 在 UML 中使用**“木头人”**  表示参与者，名字放在图标下侧，具有属性和操作

## （三）用例图（Use Case Diagram）

1. 定义及性质

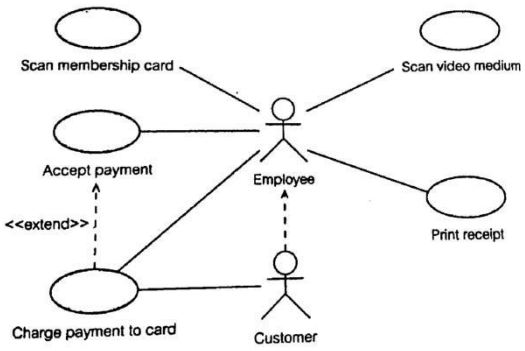
(1) 参与者和用例的可视化表示，不存在孤立节点

(2) 将用例赋给参与者，允许用户在用例之间**建立关系**，相同的参与者可以出现多次

(3) 带有主题的建模需将参与者放在主题框的**外围**

① A -----<<**extend**>> -----> B：A 对 B 的功能进行扩展

② A -----<<**include**>> -----> B：A 总是包含 B




## （四）用例文档化（Documenting use cases）

1. 用例文档的结构

(1) 简要描述

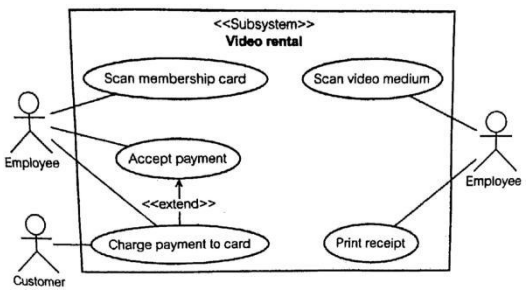
(2) 涉及的（involved）参与者

(3) 前置条件（preconditions）——用例开始所需要的条件

(4) 事件流的详细描述

① 主事件流（Main flow）& 子事件流（Sub-flow）

② 备选流（Alternative flows）——定义异常情况




(5) 后置条件（Post-conditions）——定义用例结束后的系统状态

# 二、活动视图（Activity View）

## （一）活动建模（Activity modeling）

### 1. 活动模型的定义

(1) 能够可视化地表示用例中动作的顺序（类似于程序流程图）

(2) 表示某一个用例的行为，由独特的元素组成，描述用例内部的活动细节

(3) 填补用例模型中系统行为的高层表示与交互模型（顺序图和通信图）中行为的低层表示之间的空隙

## （二）动作（Action）

### 1. 动作的定义

(1) 将活动的执行步骤称为动作，在一个活动内，动作不能被进一步分解

(2) 动作可以从用例文档中建立

(3) 在 UML 中用圆角矩形 ▭ 表示动作

## （三）活动图（Activity diagram）

### 1. 活动图的定义

(1) 活动图描述哪些步骤可以顺序执行、哪些步骤可以并行执行，显示计算的步骤

(2) 本质——带有并发功能的程序流程图

(3) 活动图显示连接动作和其他节点（如决策、分叉、连接、合并和对象节点）的流

① 分支（branch）及合并（merge）——产生可选（alternative）的计算线程（thread）

② 分叉( fork/bifurcation )及交汇(rejoin/intersection)——产生并发( co-current/parallel )的计算线程

③ 从一个动作到下一个动作控制的流程称为控制流

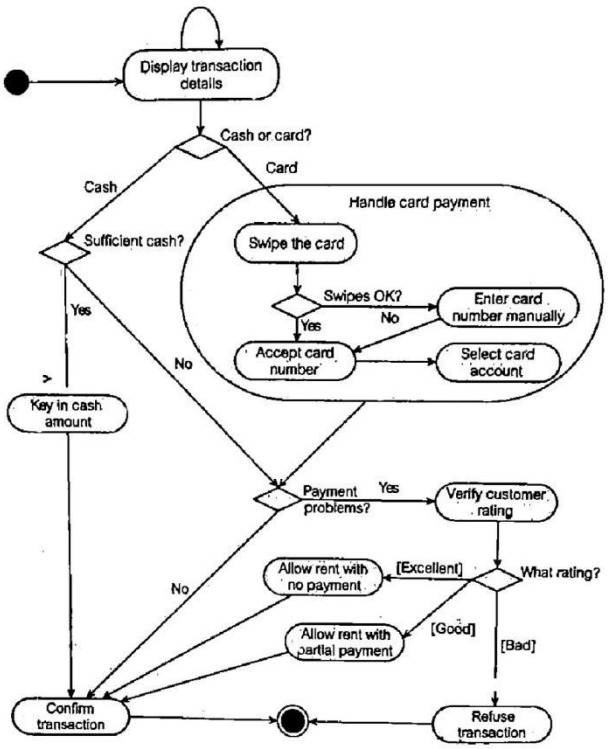
### 2. 活动图的表示

(1) 实心圆●——表示活动的开始

(2) 钻石框◆——表示分支条件，出口由事件（如 Yes，No）或守卫条件（如[green light]）控制

(3) 短线—（bar line）——表示流的分叉或连接

(4) 牛眼◉——表示活动的结束



# 三、结构视图（Structure View）

## （一）结构视图

### 1. 定义及性质

(1) 描述程序的**数据结构**以及他们的关系，比如类图的实体类图，实体类建模**必须**在需求分析阶段解决

(2) 表示系统的**静态**视图——表示数据结构、**数据关系**、作用在这些数据上的操作

## （二）类（Class）

### 1. 类的分类

(1) 持久化类

① 实体类（模型类）——即**业务对象**，表示持久的数据库对象，如订单、运输货物

(2) 非持久化类

① 表现（边界、视图）类——定义 GUI 对象，如 GUI 屏幕表单

② 控制类——控制程序逻辑及处理用户事件

③ 资源类——与外部数据进行交互和通信

④ 中介者（调解）（mediator）类——管理内存高速缓存中的实体对象的责任赋予了另一种类

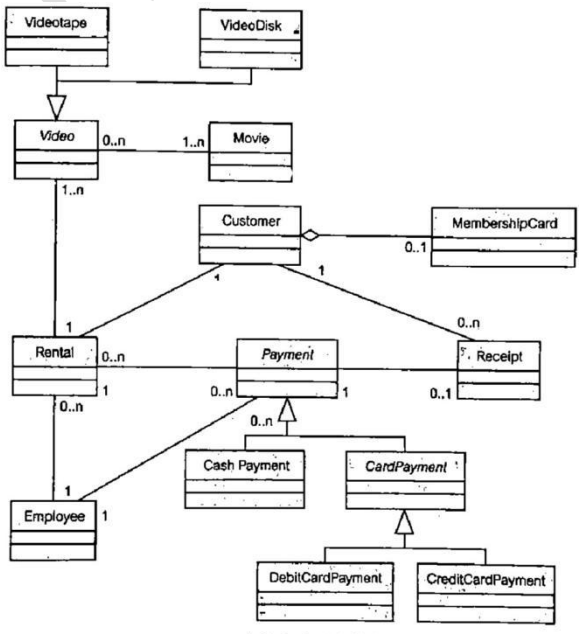
### 2. 如何判断一个类

(1) 是否为数据的容器？

(2) 是否具有取不同值的独立属性？

(3) 是否有很多实例对象？

(4) 是否在应用领域的范围（边界）内？




音像商店系统的类图

## （三）类图（Class diagram）

### 1. 定义及性质

(1) 静态建模的主要可视化技术，是主要的结构图

(2) 将类建模的所有元素的可视化表达组合在一起

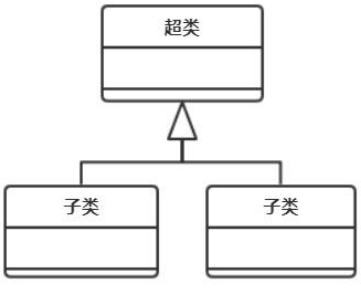
(3) 类图中的类包含属性，但是**没有操作**

### 2. 类之间的关系

(1) 关联（Association）——用关联线表示

(2) 聚合（Aggregation）——关联的一种形式，用一端带有钻石装饰的关联线（聚合：◇，复合：◆）

(3) 泛化（Generalization）——关联的一种形式，用带有大的空三角形的实现表示泛化，大的空三角形附在超类一端




仅供学习参考

# 四、交互视图（Interaction View）

## （一）交互视图

1. 交互视图的定义和性质

(1) 捕获对象之间的**交互**，为了执行一个用例或用例的一部分，这些对象之间需要通信

① 交互——是某种行为的消息集合，这些消息在连接上的角色之间进行交换

(2) 创建交互视图的**前提条件**

① 完成用例视图

② 完成类模型

③ 完成活动图

(3) 显示了协作对象之间的**事件（消息）**顺序

(4) 用于需求分析的较高级**（中后期）**阶段

(5) 表示用例的实现，比活动图更具象，趋向于对用例的某些部分（活动图中的单个活动）的建模，

## （二）顺序图（Sequence diagram）——对象交互过程的**时间**顺序

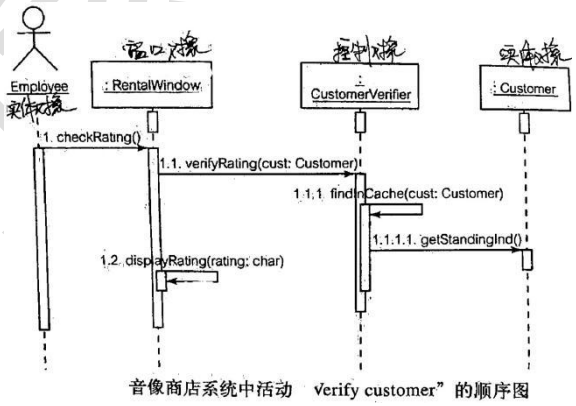
1. 顺序图的组成

(1) 水平维度——角色（对象、类、接口等）

(2) 垂直维度——消息序列，显示消息的顺序

(3) 垂直虚线——对象的生命线

(4) 垂直的高矩形——**激活**（或执行规格说明 execution specification），在生命线上被激活的方法



音像商店系统中活动 Verify customer" 的顺序图

(5) 箭头——表示一条消息，从调用对象发给被调用对象，每条**消息**实际是被调用对象的一个**方法**

(6) 星号——迭代标记，标记在消息标签前，表示在收集上的迭代

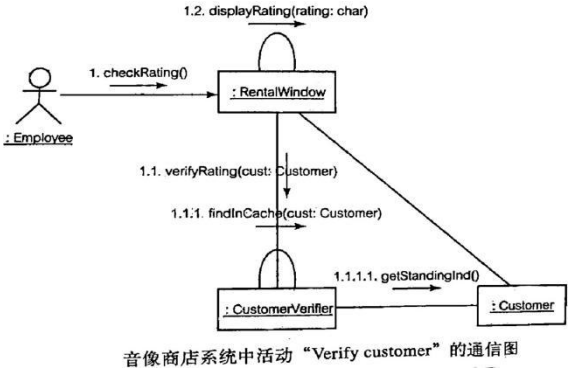
## （三）通信图（Communication diagram）——对象之间的**协作**关系

1. 通信图的性质

(1) 在通信图中没有生命线或激活，但两者都隐含地以箭头表示消息

(2) 一般情况下，当表示设计很多对象的模型时，通信图比顺序图更形象

(3) 通信图是顺序图的另一种表示方法



音像商店系统中活动 "Verify customer" 的通信图

## （四）类方法

1. 消息与方法的关系

(1) **消息**对应类图中的**方法**，消息与方法间一一对应

(2) 每一条消息触发被调用对象上的一个方法，操作与消息具有相同的名字

(3) 顺序图中的每一条消息是起点对象调用终点对象的一个类方法，作为被调用对象的一个类方法

# 五、状态机视图（State machine view）

## （一）状态机模型

### 1. 定义和性质

(1) 说明类中的**动态变化**，描述类的对象可能具有的不同**状态**和不同状态的**改变**

(2) 捕获类的**生命历史**

(3) 状态机模型是为具有**感兴趣**的状态变化的类构造的，而不是为任何状态变化的类构造的

## （二）状态机图

### 1. 定义和性质

(1) 状态图描述某一个类或一个对象不同状态的**改变**

(2) 状态图是**状态**和由**事件**引起的**转换**的偶图

(3) 状态图是**业务规则**模型

① 业务规则：在一段时期内是保持不变的，相对独立于特定的用例

### 2. 状态机图的组成元素

(1) 实心圆●——表示活动的开始

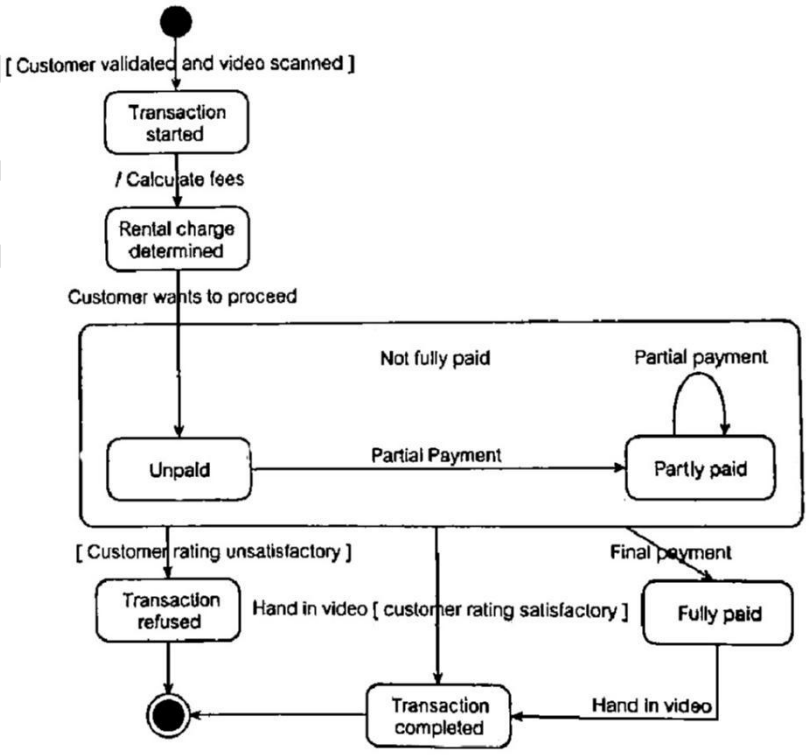
(2) 圆角矩形▭——一个状态

(3) 箭头→——状态的转换

① 转换语言：Event (parameters) [guards] / action

1) **守卫**和**事件**的区别：在事件的处理点上估算守卫条件来决定是否将被触发

(4) 牛眼◉——表示活动的结束




音像商店系统中Rental类的状态机图

# 六、实现视图（Implementation View）

## （一）子系统和包

### 1. 子系统

(1) 子系统是系统的一部分，**封装**了一部分系统行为

(2) 子系统提供的服务是由其**内部**的组成部分（它的类）所提供的服务的结果

(3) 子系统可以在体系结构层被**结构化**，可以包含其他子系统，但是**不能被实例化**

(4) 子系统与子系统通过**接口**连接

### 2. 包

(1) 包是具有指定名字的建模元素的**分组**

(2) 包可以包含其他包，拥有其成员，可以直接被映射到程序设计语言元素结构，**不暴露接口**

### 3. 区别与联系

(1) 区别

① 子系统是一个**物理实体**（元件），能够实现一定的功能；包是子系统的**逻辑部件**（仓库）

② 对于子系统，客户请求**子系统本身**完成行为；对于包，客户请求**包内的某元素**完成行为

③ 接口对子系统的调用使用**<<implementation>>**；对包的调用使用**<<use>>**

(2) 联系

① 子系统是一个**更丰富**的概念，既包含包的结构化方面，又包含类的行为方面，行为由一个或多个接口提供，客户通过这些接口请求子系统的服务

## （二）构件和构件图

### 1. 构件（Component）

(1) 构件 = **模块化**（modular） + **可部署**（deployable） + **可替换**（replaceable）

(2) **提供（provided）接口**：由构件实现了的接口，表示构件的实例为它们的客户所提供的服务

(3) **依赖（required）接口**：可能被构件使用的接口，指明一个构件为了执行它的功能并完成它自己向客户提供的服务所需要的服务

### 2. 构件图

(1) 构件——带有**<<component>>**关键词标记的矩形

(2) 提供接口——附在构件上的小圆O，用直线连接表示构件的矩形和小圆

(3) 依赖接口——附在构建上的半圆⌒，用直线连接表示构件的矩形和半圆

(4) 依赖线——虚线箭头- - ->，由某一个依赖接口指向一个提供接口

## （三）节点和部署图

### 1. 节点

(1) 节点是**人工制品**可以在上面部署运行的计算资源，可以将节点通过通信路径连接起来定义网络结构

仅供学习参考

① 人工制品：物理信息块的规格说明，由软件开发过程或系统的部署和运行使用或生成（如模型文件、源文件、脚本、二进制可执行文件、数据库系统中的表、可交付的部署、邮件信息等）

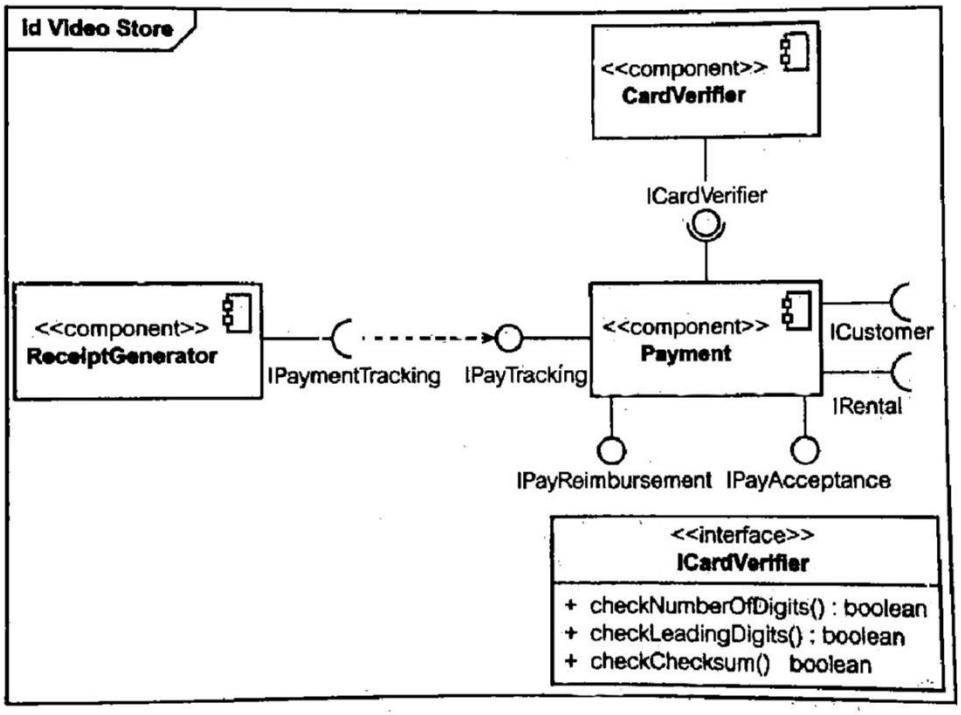
2. 部署图

(1) 关注结构和节点依赖建模，节点定义系统的实现环境

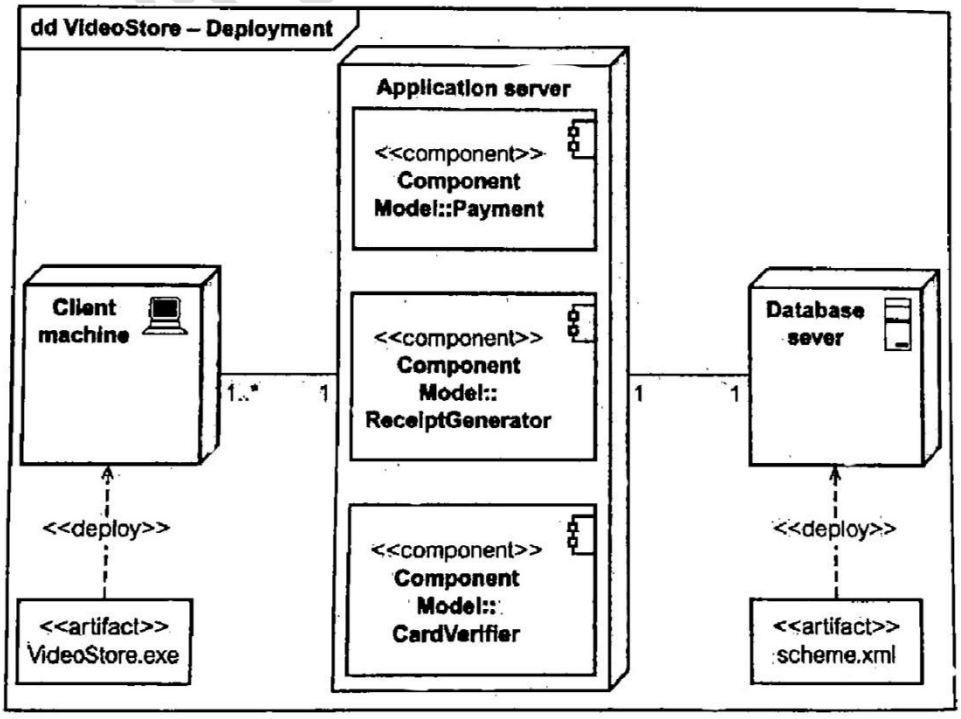
(2) 立方体表示部署图中的一个节点，用带<<deploy>>标识的虚线箭头表示人工制品到节点的部署关系

(3) 通过关联将节点连接起来表示通信路径

(4) 关联端的重数值表示在关联中涉及的节点实例的数量




音像商店系统的构件图




音像商店系统的部署图